周暐倫 Terry Chou ・ 國立中央大學資訊工程碩士

作品附件

履歷上的九個專案加上目前開發中的遊戲 Cyclepact,每案一頁,含畫面截圖、規模數字表與設計取捨。四個 live 站台由本人自架維運(Oracle Cloud ARM + Cloudflare Tunnel,GitHub Actions 自動部署)。

● AI-ML 部署　● 全端 / 系統　● DevOps / SRE

	專案	方向	期間	線上 / 原始碼
學術專案				
01	**智慧導航系統** Smart Navigation System	● AI-ML ・ 電腦視覺	2022/9 – 2023/6	GitHub
02	**AI 模型部署與監控平台** AI Model Deployment & Monitoring Platform	● AI-ML 部署	2024/11 – 2025/6	GitHub
個人專案 ・ Side Projects				
03	**製造業 ERP 系統** Manufacturing ERP	● 全端 ・ 後端	2026/6 – 至今	erp.terrychou.com
04	**mcpglass — MCP 流量觀測與安全代理** mcpglass — MCP traffic observability & security proxy	● DevOps ・ 可觀測性	2026/7	GitHub
05	**ServerMonitor 遊戲伺服器管理工具** ServerMonitor	● DevOps ・ SRE	2026/7	GitHub
06	**AI 用量監控 AI Usage Monitor** AI Usage Monitor	● 系統程式 ・ 桌面	2026/7	GitHub
07	**Steam 特價追蹤站** Steam Sale Checker	● 全端 ・ 維運	2026/6	steam.terrychou.com
08	**美食推薦地圖「吃什麼?」** FoodMap — "What to Eat?"	● 全端 ・ 維運	2026/8	eat.terrychou.com
09	**魂晶獵手 Soulshard Hunter** Soulshard Hunter	● 前端 ・ 遊戲	2026/4 – 至今	soulshard.terrychou.com
10	**輪迴盟約 Cyclepact** Cyclepact	● 遊戲 ・ Unity	2026/6 – 至今	私有 repo ・ 開發中



AI 模型部署與監控平台



智慧導航系統



製造業 ERP 系統



mcpglass — MCP 流量…



ServerMonitor 遊戲伺服…



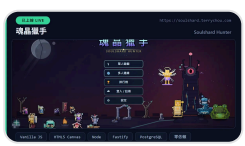
AI 用量監控 AI Usage M…



Steam 特價追蹤站



美食推薦地圖「吃什麼?」



魂晶獵手 Soulshard Hu…



輪迴盟約 Cyclepact

01 / 10 **大學專題** 2022/9 – 2023/6

智慧導航系統

Smart Navigation System



作品卡

定位	大學專題:用 ROS 2 小車模擬車輛、手機當導航,以街景語意分割找出前方可行駛的區域、以手勢辨識偵測路邊有人招手並當作停車信號,再把導航的轉向、距離與路名疊加在畫面上。
期間	2022/9 – 2023/6
技術	Python, ROS 2, TensorFlow 1.x, FC-DenseNet103, OpenCV, MediaPipe, NumPy, Pillow, Android (Java), HERE SDK, Java NIO Socket, gTTS / playsound
規模	三模組:Python 主程式 · Android(Java) · MediaPipe 手勢 CamVid + BDD100K 1:1 混合訓練(只用兩邊共有的類別) FC-DenseNet103 語意分割(TensorFlow 1.x) 燈號辨識最終版未實作(README 明載)
連結	github.com/q86865511/Smart-Pedestrian-Navigation-via-Scene-Analysis-and-Traffic-Light-Detection

做了什麼、怎麼做的

- 以 FC-DenseNet103 在 TensorFlow 1.x 上訓練街景語意分割,採 CamVid 與 BDD100K 1:1 混合,只使用兩邊共有的類別(剛好涵蓋導航需要的類別);附帶 label 顏色轉換工具,讓兩套資料的標註對齊。
- 在分割結果上以 BFS 連通域分析計算路面區塊的形心與面積,判斷小車前方可行駛的區域,並把手機導航的距離、轉向角、路名與導引箭頭即時疊加回畫面,而不是只輸出一張遮罩。
- 用 MediaPipe Hands 加上 keypoint / point-history 分類器辨識路邊有人招手的「Stop」手勢,連續觸發時視為停車信號,並以語音(gTTS 預生成、playsound 播放)提示。
- 跨裝置整合:手機端導航 App(改自 FancyNavi,基於 HERE SDK)提供大方向,透過自寫的 Java NIO 非阻塞 Socket 伺服器把路線資訊傳給小車端;TCP 是位元組串流,接收端為每條連線保留緩衝、只解碼完整的一行,避免中文路名被切半而解碼失敗。

02 / 10 **碩士論文** 2024/11 – 2025/6

AI 模型部署與監控平台

AI Model Deployment & Monitoring Platform



作品卡

定位	為 YOLOv8 系列模型打造的端到端部署平台:自動化 PT→ONNX→TensorRT 優化、多批次多精度效能評測;以 Triton 完成模型上架與生命週期管理,效能量測走本機 Ultralytics 路徑,並以 Prometheus/Grafana 即時監控。
期間	2024/11 – 2025/6
技術	Python, FastAPI, asyncio, React, Ant Design, YOLOv8, ONNX, TensorRT, Triton, OpenCV, Docker, Prometheus, Grafana
規模	Python 7,875 行 ・ 前端 6,092 行 Docker Compose 9 服務(3 個宣告 GPU) 支援 yolov8 / pose / seg;FP32 / FP16 Prometheus 4 個 scrape job ・ Grafana 4 面板
連結	github.com/q86865511/AI-Deployment-Pipeline

做了什麼、怎麼做的

- 自動化模型優化管線:以 ultralytics 與 trtexec 完成 PT→ONNX→TensorRT 轉換,並對 FP32/FP16 × 多組批次大小(預設 1/2/4/8/16)各做多次重複推論(每組上限 10 次),同時量測 mAP50/mAP50-95 準確度與延遲、FPS、GPU 使用率、VRAM,並輸出平均值與標準差以評估穩定性。
- 以 NVIDIA Triton Inference Server(explicit model-control 模式)管理模型上架、動態掛載/卸載與即時推論統計,效能量測則走後端容器內的 Ultralytics in-process 路徑;結果分析頁可多維度比較不同精度與批次配置,並匯出 PDF/Excel 報告作為選型與硬體規劃依據。
- 設計清晰分層的服務架構:FastAPI 非同步後端(轉換/推論/評測/Triton 各自獨立 service 與 router)+ React/Ant Design 前端 + NVIDIA 推論堆疊,職責邊界明確,易於擴充與除錯。



轉換設定頁(demo 影格)



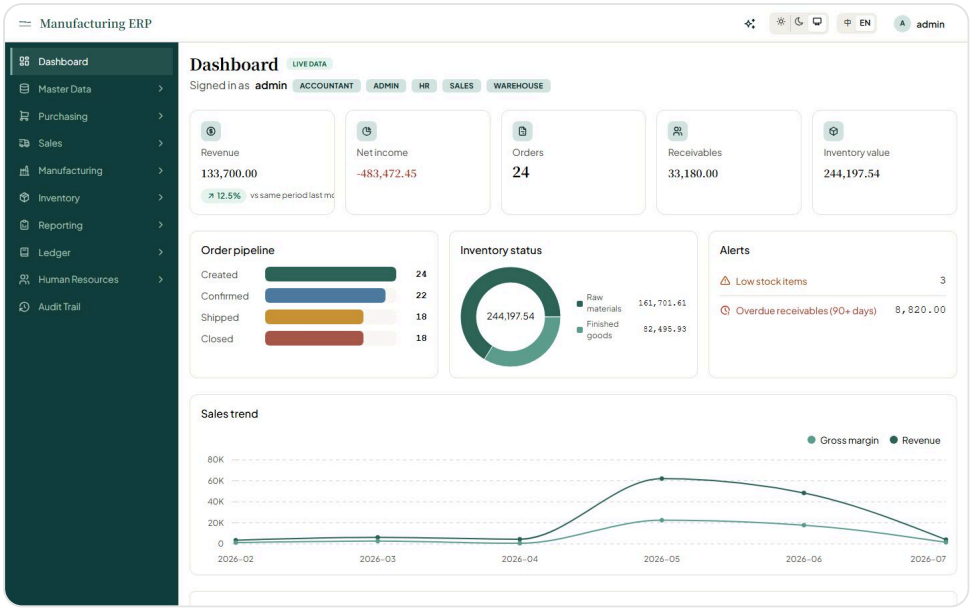
結果報告頁(demo 影格)

03 / 10　2026/6 – 至今

LIVE

製造業 ERP 系統

Manufacturing ERP

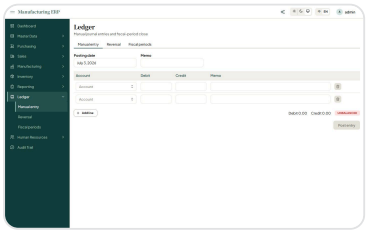


Dashboard

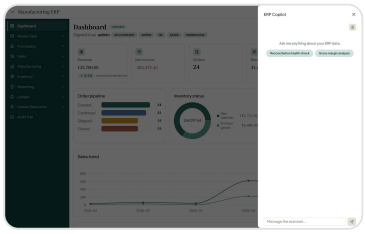
定位	從零打造、已上線可實機體驗的全端製造業 ERP,涵蓋採購、生產、銷售、付款到人資的完整營運流程;核心是手寫的複式記帳總帳引擎——所有業務動作一律以借貸平衡的分錄入帳,帳永不半套、不平不可寫入。
期間	2026/6 – 至今
技術	Java 21, Spring Boot 4, PostgreSQL 16, Spring Data JPA, Flyway, Spring Security / JWT, JUnit 5, Testcontainers, ArchUnit, React 19, TypeScript, Mantine 9, Vite, TanStack Query, Vitest, Playwright, springdoc-openapi, Claude API / MCP, Micrometer / Prometheus, nginx, Docker Compose, GitHub Actions
規模	測試 649:單元 135 + 整合 239(Testcontainers)+ 前端 246 + MCP 29 13 個後端模組 ・ ArchUnit 26 條邊界規則 Java 16,750 行 ・ 前端 TS/TSX 22,807 行 Flyway 24 個 migration ・ CI 6 個 workflow ・ JaCoCo 77%
連結	**erp.terrychou.com** ・ github.com/q86865511/ERPSystem

做了什麼、怎麼做的

- 手寫複式記帳總帳引擎:進貨、生產、出貨、付款等業務動作,與其單據、庫存異動在同一筆資料庫交易內原子過帳成借貸平衡分錄;debit=credit 與「已過帳不可修改/刪除」更下放為資料庫約束,從資料層根本杜絕半套帳與不平帳。
- 模組化單體 + 機械化架構驗證:13 個後端模組(總帳、庫存、採購到付款、訂單到收款、製造、人資等)只能透過彼此的 published port 互動;邊界由 ArchUnit 在 CI 強制——違規 import 他模組內部或直接動其資料表就會讓建置失敗,邊界是測試而非口頭約定,設計取捨記錄於 ADR-0001。
- 金額正確性與真實整合測試:自訂 Money 值物件(BigDecimal、scale 4、HALF_UP,全程不用 float/double);以 JUnit 5 + Testcontainers 對真實 PostgreSQL 16 跑整合測試(非 mock)——後端 135 單元 + 239 整合共 374 全綠、前端另有 246 個 Vitest 測試(JaCoCo 覆蓋率 77%),借貸平衡、不可竄改等不變式以實際資料庫約束驗證。



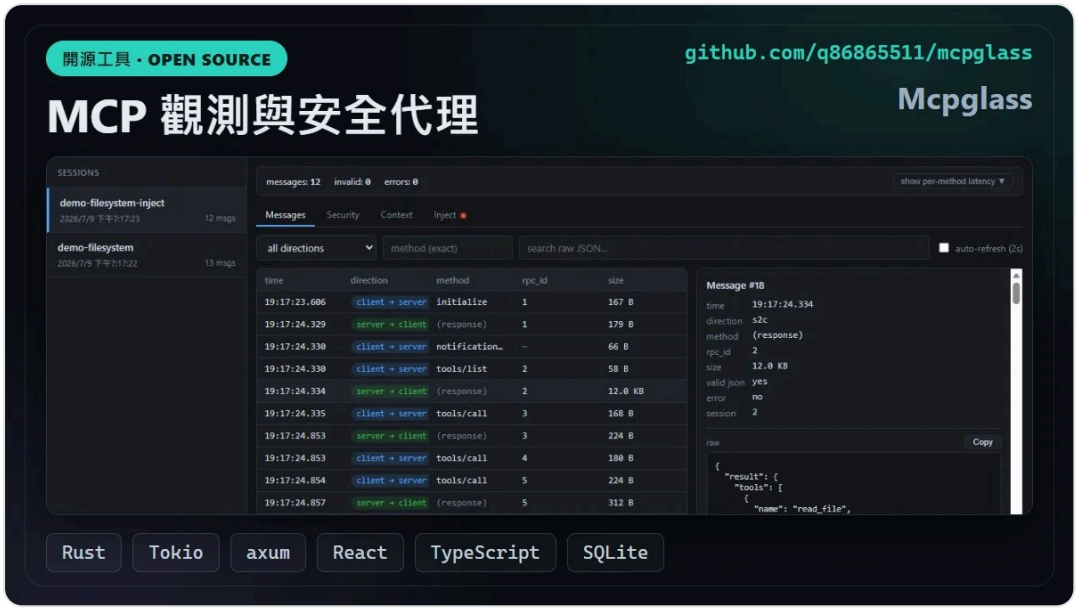
複式記帳總帳



ERP Copilot(AI 助手)

mcpglass — MCP 流量觀測與安全代理

mcpglass — MCP traffic observability & security proxy

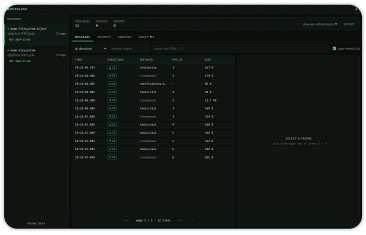


作品卡

定位	MCP 流量的 Wireshark＋防火牆——單一 Rust binary 透明代理,坐在 AI client(Claude Code/Cursor…)與 MCP server 之間(stdio 與 Streamable HTTP 皆支援):錄下每一幀 JSON-RPC、偵測工具描述被偷改的 rug-pull 與外流密鑰、估算工具 schema 的 context 成本,還能重放請求與注入故障;fail-open 鐵律保證代理自身故障絕不阻斷流量,資料 100% 留本機(單一 SQLite 檔＋僅綁 loopback 的儀表板)、零遙測;Apache-2.0 開源,CI 於 GitHub Actions。
期間	2026/7
技術	Rust, Tokio, axum, React, TypeScript, SQLite, GitHub Actions
規模	約 260 個測試(237 同步 + 25 async) Rust 19,206 行(30 檔)· 前端 4,230 行、零 UI 框架 CI 4 個 workflow:Linux / macOS / Windows matrix、release 含 SBOM + provenance SQLite 5 張表 schema v7 · 全 repo 零 TODO
連結	github.com/q86865511/mcpglass

做了什麼、怎麼做的

- fail-open 鐵律架構:代理自身任何錯誤(解析失敗、DB 寫入失敗、policy panic)都不得阻斷或延遲 client↔server 流量——s2c 方向是「先轉發、後記錄」的旁路 tap,c2s 方向是同步純函式決策;記錄通道 try_send 不施加背壓、channel 滿時日誌節流避免故障態的同步 I/O 回壓 wire,底線由 bytes 完全一致的 passthrough 整合測試守住。
- MCP 供應鏈安全:tools/list 回應以自實作的 canonical-JSON key 排序 SHA-256 指紋,綁 server 啟動指令為跨 session 身分,抓「核准後偷改工具描述」的 rug-pull(含 A→B→A 震盪偵測);secret 外流偵測內建八家密鑰 pattern;預設 monitor 不阻擋,enforce 命中時合成協定內 -32001 錯誤回 client——是合法拒絕,不是斷線。
- 兩種 transport、一條管線:stdio wrap 與 Streamable HTTP gateway(SSE 逐 chunk 直通+旁路切分、非 SSE 回應緩衝上限、Origin+Host 雙重 loopback 驗證防 DNS rebinding/CSRF-to-localhost)共用同一 tap/storage/指紋管線;attach 一鍵改寫三家 client 設定檔,原子寫入+備份+冪等+detach 精確還原,url server 的原始 upstream 記在 gateway.toml 作唯一真相。



儀表板總覽

ServerMonitor 遊戲伺服器管理工具

ServerMonitor

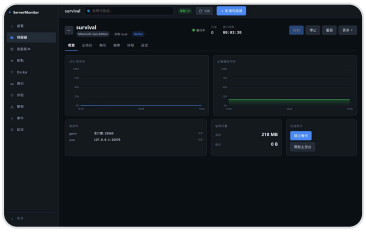


Dashboard

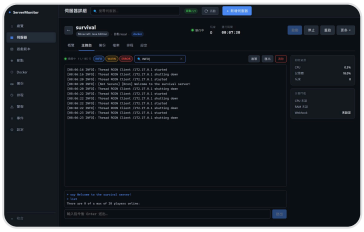
定位	可擴充的遊戲專用伺服器管理與監控桌面工具——範本驅動(新增遊戲=一份 TOML 檔)、Docker/原生 Windows 雙執行後端、多節點遠端管理,在單一 GUI 內涵蓋建立、監控、指令、備份、自動重啟到告警的完整營運循環。
期間	2026/7
技術	Go, Wails, Svelte, TypeScript, Docker, SQLite
規模	Go 測試 550 個(89 檔;CI 跑 532) Go 26,464 行 • 測試 22,894 行(比 0.87) Svelte 10,446 行 + TS 1,265 行 內建模板:Minecraft(4 變體)、Palworld
連結	github.com/q86865511/ServerMonitor

做了什麼、怎麼做的

- 範本驅動的擴充性:帶版本的 TOML schema 宣告變體/loader、埠衝突鍵、參數、機密、RCON/REST 指令協定、健康探針與生命週期 hook,新增遊戲=加一份範本檔;內建範本以 go:embed 嵌入執行檔,自訂範本熱載入並經驗證拒載。
- 同一 RuntimeBackend 介面下的雙執行後端:Docker 容器,或全自動供應(Adoptium JRE、五種 Minecraft loader、SteamCMD)的原生 Windows 行程,以 Job Objects 強制資源上限,agent 側將 OOM 與一般 crash 分流回報;備份為停機一致 tar 快照 +checksum,格式在 docker/native 兩後端間互通,agent 重啟自動收養仍在跑的行程。
- 天生多節點:核心對本機也走 HTTP 節點代理,遠端 Linux 節點只需一個靜態執行檔——首啟自動生成持久 token 與自簽 TLS 憑證,GUI 以 SHA-256 指紋 TOFU 釘選,憑證被換直接拒連;節點 token 一律入 OS 金鑰庫。



伺服器詳情



主控台

AI 用量監控 AI Usage Monitor

AI Usage Monitor



作品卡

定位	以 Tauri 2(Rust + React)打造、常駐 Windows 系統匣的 AI CLI 用量儀表——同時監控 Claude Code 與 Codex CLI 的 5 小時/週視窗額度,托盤圖示本身就是雙環量表;官方數字優先、資料源失效時優雅退化為明確標示的本機估算,跨門檻主動通知、燒速異常預警,100% 本機處理零遙測。
期間	2026/7
技術	Rust, Tauri 2, React, TypeScript, Vite, SQLite, tiny-skia, SMTP
規模	Rust 測試 133 個 ・ Rust 7,204 行(測試約 3,080) 前端 2,599 行 ・ 安裝檔約 4.7 MB(NSIS) SQLite 90 天用量時間序列;120 秒節流官方查詢 托盤雙環:≥80% 黃、≥95% 紅
連結	github.com/q86865511/UsageMonitor

做了什麼、怎麼做的

- 官方數字優先的穩健資料狀態機:Codex 直接解析 session 檔內伺服器端 rate_limits 事件、Claude 走 OAuth 用量端點,都是帳號級準確百分比而非本機猜測;失敗一律以 HTTP 狀態分類(401 才是真過期)、429 尊重 Retry-After 退避、暫時性失敗沿用最後成功值並標示「更新於 N 分前」,只有真失效才退回本機統計且明標「估算值」——資料源不可用就優雅退化,絕不謊報。
- 檔案監看+增量解析,幾百 MB 歷史也不拖慢啟動:notify 監看兩個資料目錄(2 秒 debounce+60 秒兜底輪詢);Claude 對話紀錄以 offset 續讀、message.id 去重、mtime 剪枝與游標修剪,只重讀有變動的檔尾;設定變更(含資料目錄路徑)熱重載即時生效,每家資料源各有獨立的手動刷新。
- 純函式通知規則引擎:用量門檻、額度回滿、燒速異常(依 usedPercent 斜率預估「將在 N 分鐘內打滿」,抓得住失控的 agent 迴圈)三類規則,通道可選 Windows toast/SMTP Email/兩者;同一視窗每條規則只觸發一次、resets_at 重置後重新武裝,武裝狀態跨重啟持久化、重開 app 不補發;Email 走獨立發送執行緒,內建防風暴閘門(失敗退避+每小時上限)與寄測試信。



托盤儀表(實際截圖)

LIVE

Steam 特價追蹤站

Steam Sale Checker

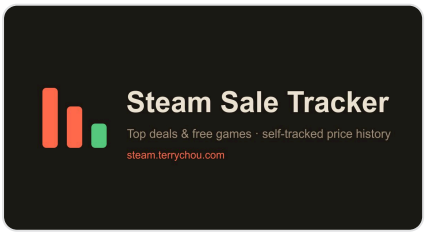


docs 封面

定位	自建並已上線的 Steam 特價追蹤站——SteamDB 風的特價榜 + 自建價格歷史 + Discord 降價通知:免登入即可瀏覽特價與免費遊戲,Discord 登入後解鎖跨裝置願望清單與目標價提醒。
期間	2026/6
技術	TypeScript, Astro, Node.js, Fastify, SQLite, Discord OAuth, Docker, Caddy, Cloudflare Tunnel, GitHub Actions, Vitest
規模	374 個測試 / 31 檔(worker 196、api 90、web 52、shared 36) TS 4,295 行 + 測試 2,810 行(比 0.65) SQLite 14 張表 ・ 特價榜上限 120 ・ 價格歷史 365 天 抓取間隔 1800 秒 ・ Steam 最小請求間隔 1.5 秒
連結	**steam.terrychou.com** ・ github.com/q86865511/SteamSaleChecker

做了什麼、怎麼做的

- 自建價格歷史而非串接第三方:worker 每輪把抓到的台幣價寫進 SQLite,累積成「本站追蹤以來」的走勢與史低(observed low),並誠實標示為『追蹤以來最低』而非冒充 SteamDB/ITAD runtime;另可選配 ITAD API Key 以真實台幣史低校正冷啟動初值,只改 game_stats、不污染累積曲線。
- 「server 端抓取、前端純靜態 JSON」的架構:對 Steam / GamerPower / ITAD 的呼叫全在 worker 完成,烤成 deals / free / meta 給前端 fetch,公開瀏覽零金鑰外洩、免重 build;npm-workspaces monorepo 切成 shared 純函式+型別 / worker / Astro web / api,純邏輯以 vitest TDD(全專案 374 個測試)。
- 可登入的通知子系統:Discord OAuth 登入後願望清單跨裝置同步(未登入 localStorage、登入即合併)、每款可設目標價;worker 偵測「收藏且創本站新低/跌破目標」時,由 Discord bot 依 per-user 偏好以頻道 @你或私訊 DM 發送,含個人免費遊戲通知與每日/每週特價摘要;另可一鍵從公開 Steam 願望單(貼 SteamID64 或個人檔網址、免金鑰)匯入收藏。



站台分享圖

LIVE

美食推薦地圖「吃什麼?」

FoodMap — "What to Eat?"



首頁:依類型 / 城市 / 人數 / 場合挑店

定位	自建並已上線的個人美食名單網站:只收站主吃過或想吃的店,依類型、城市、人數與場合幫你挑一家,並在地圖上標出位置。
期間	2026/8
技術	Node.js, Express, React, Vite, SQLite, Leaflet, Docker, Caddy, Cloudflare Tunnel, GitHub Actions
規模	收錄 157 家店(2026-08)・ 分類存資料表,新增不改程式 SQLite(Node 內建 node:sqlite)10 張表 規則式推薦為主;Gemini 選配,只能從名單挑、失敗退回規則式 Docker 多階段 build ・ GitHub Actions 部署 + 健康檢查
連結	**eat.terrychou.com** ・ 原始碼為私有 repo(不公開)

做了什麼、怎麼做的

- 規則式推薦為主、AI 為選配:依類型、城市、人數與場合篩選並排序;設定 Gemini 金鑰才開放自然語言提問,且 AI 只能從站主名單挑選、回傳結構化結果,失敗時退回規則式並標明來源。
- 資料模型為擴充與篩選設計:分類存在資料表,新增分類不改程式;價位同時存原文與解析後的數字,畫面顯示原文、篩選排序用數字。
- 選用 Node 內建的 SQLite 而非原生編譯套件,讓 ARM 主機上的容器建置更單純;Docker 多階段 build,服務只綁本機埠,由 Caddy 與 Cloudflare Tunnel 對外。



口袋名單



店家頁(查證來源標註)



推薦條件

09 / 10　2026/4 – 至今

LIVE

魂晶獵手 Soulshard Hunter

Soulshard Hunter

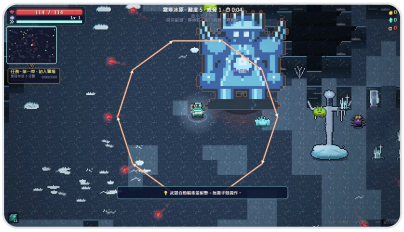


docs 封面

定位	用原生 HTML5 Canvas、零建置、零 npm 執行期相依打造的像素風 roguelike 生存遊戲(對外版本 V2.0):sprite 100% 程式即時生成,63 敵人 / 43 武器 / 27 角色 / 10 生態系的內容規模,並可選配雲端帳號與 1~3 人即時連線合作。
期間	2026/4 – 至今
技術	JavaScript, Node.js, Fastify, PostgreSQL, WebSocket, Docker, GitHub Actions, Oracle Cloud
規模	JS 39,728 行(179 檔)・ 768 個 sprite 定義 63 敵人 / 43 武器 / 27 角色 / 10 生態系 / 219 成就 後端 185 個斷言 + 前端 59 條 Playwright smoke 作部署雙關卡 12 首實錄配樂 ・ V2.0(Round 29)
連結	**soulshard.terrychou.com** ・ github.com/q86865511/Soulshard-Hunter

做了什麼、怎麼做的

- 從零自製遊戲引擎:120 Hz 固定步主迴圈降低輸入延遲、空間網格取代 O(n^2) 鄰近查詢(260 敵人上限仍穩定)、自寫相機與世界↔螢幕座標轉換、純程式繪製的像素 sprite 系統(rasterise 後快取),連音效都以 WebAudio 即時合成。
- 伺服器權威排行榜:後端依 kills/stage/time/difficulty/reaper 重算分數並忽略客戶端宣稱值,搭配逐欄位上限與「依存活時間推算的合理性閘門」反作弊(如擊殺數受時間上限約束、無盡模式不可能觸發破關或死神),從源頭杜絕分數灌水;客戶端另有誠信閘門——啟用輔助模式或動用開發者面板的對局一律不上傳。
- 主機權威中繼的 1~3 人即時合作:開房玩家瀏覽器跑權威模擬並以 ~18Hz 廣播量化後的世界快照,Node 伺服器只當房間/中繼不跑模擬;單一 protocol.js 同時擁有編碼與解碼以防漂移,並對未信任的 host 資料做夾擠防範記憶體攻擊。單機路徑完全不受影響。



Boss 預警(遊戲畫面)



天賦介面

10 / 10 **開發中** 2026/6 – 至今

輪迴盟約 Cyclepact

Cyclepact



戰鬥畫面(第一章實機截圖)

定位	開發中的回合制劇情向收集養成 RPG(Unity 6 / C#):以「盟約」取代捕捉、主線為時間輪迴,第一章已可從新遊戲玩到章末。
期間	2026/6 – 至今
技術	Unity 6, C#, URP 2D, .NET, Python, Blender
規模	世界設定:18 屬性 · 24 區域 · 500 隻精靈 · 4 種結局 第一章(5 區 + 旅店)可從新遊戲玩到章末;第二章開發中 Core 為不引用 Unity 的純 C#,同一份測試在 dotnet test 與 Unity EditMode 都跑 精靈 / 地圖 / 對話皆為 JSON,場景由匯入器產生
連結	原始碼為私有 repo(不公開)

做了什麼、怎麼做的

- 分層架構、相依單向:Core 為不引用 Unity 的純 C# 規則與戰鬥事件引擎,上層依序是資料、地圖邏輯、地圖、美術、UI 邏輯與 UI;戰鬥 UI 把引擎發出的事件轉成逐步播放的步驟,規則與畫面完全分離。
- 資料驅動的內容:500 隻精靈、地圖與對話都以 JSON 為唯一來源,Unity 場景由匯入器產生、不手改;存檔帶版本號並有遷移器,舊存檔可升級。
- 引擎遷移:由 Godot 轉 Unity 6,主因是 Tilemap 原生支援地形自動過渡;遷移時保留舊版規則測試作為對照組,確認 C# 版規則與原版等價後才封存 Godot 版。



野外地圖



劇情對話



城鎮設計圖(碰撞資料驗證)